

Intro to Vibe Coding

Describe it. Run it. Check it. Own it.



```
> Build me an RSVP page for a  
founder dinner. 40 guests,  
mobile-first, no login. Done  
when I can RSVP from my  
phone.
```

```
building... |
```

Chai Pin Zheng · Product Engineer, Reactor School · NUS Computer Science



Who I am



Chai Pin Zheng

Year 2 Computer Science, NUS. In Ho Chi Minh City with NUS Overseas Colleges.

Product Engineer, Reactor School: I build the platform the team uses to run programmes and track alumni.

90+

hackathons

6

first-place finishes

2026

AverixHacks 1st · The Merge 2nd · DesignXR runner-up

Co-founder, Resumify. AI resume tool for ex-offenders, in partnership with Yellow Ribbon Singapore.

Community. The Collective · The LaunchPad Challenge · Sip & Scale Saigon · github.com/Ducksss



The next 60 minutes, in order

2

What it is

A definition, an analogy, and why now.

3

How to do it

Tools, the loop, and prompts that work.

4

See it

Two prompts compared, then a live build.

5

Where it breaks

Five traps and the habits that prevent them.

6

When it matters

Moving from vibe coding to vibe engineering.

7

Close

Homework, then 30 minutes of your questions.

Teaching: 30 minutes. Then 30 minutes that are yours: ask me anything, or ask me to build anything.

[1 Why](#)[2 What it is](#)[3 How](#)[4 See it](#)[5 Where it breaks](#)[6 When it matters](#)[7 Close](#)

Questions?
Scan, ask anonymously



What is vibe coding?

Describe what you want. AI drafts the software. You run it, react to it, and refine it.

Programming in natural language. You express intent, or a "vibe", and generative AI turns it into working code. You do not write the logic line by line.

“fully give in to the vibes”

Andrej Karpathy, co-founder of OpenAI, February 2025. Say stuff, run stuff, accept all, and it mostly works.



Word of the Year 2025

Collins Dictionary, nine months after the tweet.

The bottleneck moved.

It used to be: can you write the code? Now it is: can you describe what you want, and can you tell whether you got it?



Think of it as hiring a personal shopper

01

Your brief

"A relaxed outfit for a hot afternoon."

In software: "a calm one-page site for a founder dinner."

02

Their draft

They pick a first combination for you.

The AI generates a first runnable version.

03

Your feedback

"Lighter colours. Less formal."

"Simpler layout. Make the RSVP button obvious."

The shopper handles sizing, brands, and stock. The AI handles syntax, layout, and libraries. **Your taste and the final call stay yours.**



Why now



The models got good at long tasks

A whole feature, start to finish. Not one function at a time.



The agents got hands

They run your code, open a browser, read the error, and deploy.



It got cheap

A working prototype costs less than the coffee you drink while it builds.

25%

of Y Combinator's Winter 2025 startups said about 95 percent of their code was AI-written.

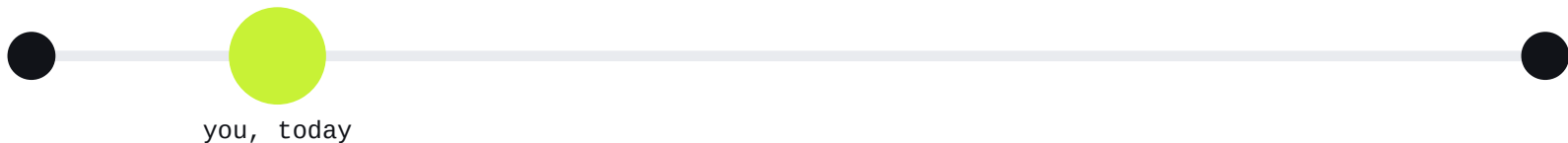
Funded companies, not hobby projects. Over a year ago.



It is a dial, not a switch

VIBE CODING

VIBE ENGINEERING



Explore a working idea.

Describe, accept, refine by hand. Good for prototypes, personal tools, weekend projects, learning.

Deliver software you can stand behind.

Specify, build, review, verify, record. Required for anything with real users, money, or personal data.

You start on the left. Move right as the thing starts to matter. We come back to the right side in Act 6.



The tools, by how much code you want to see



Never want to see code

Browser only. Prompt, preview, publish.

Lovable · Bolt.new · v0 · Replit · Google AI Studio

ChatGPT, Claude, and Gemini also build single-page apps inside the chat.



Want to see the code

An editor with an agent inside.

Cursor · Windsurf · GitHub Copilot · Kiro



Coding agents

Terminal or desktop app. Most powerful, most rope.

OpenAI Codex · Claude Code · Gemini CLI

MY DAILY STACK **Codex + Claude Code**

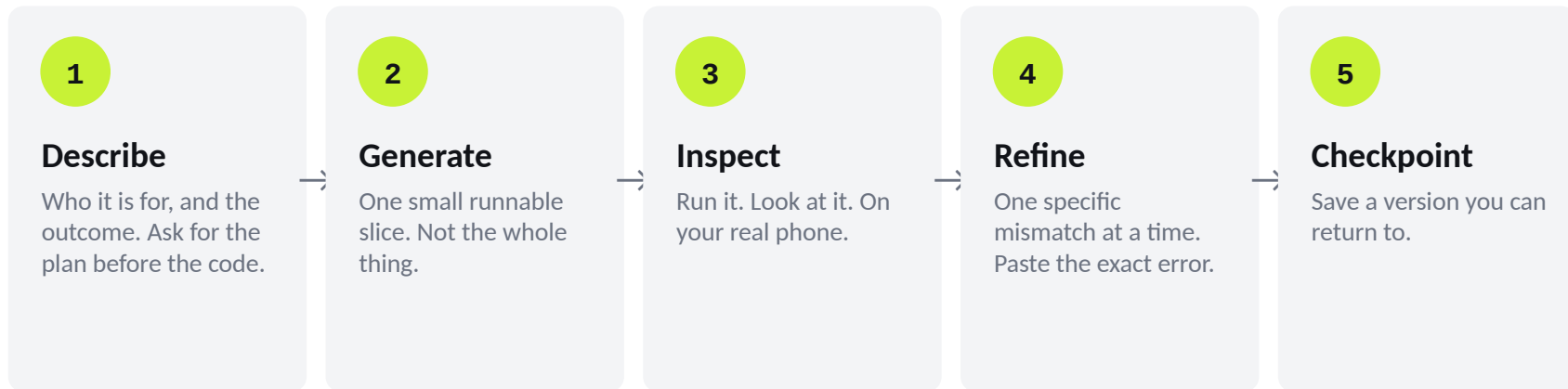
TODAY'S DEMO **Lovable**

Snapshot: Sept 2026. Changes monthly.



The loop

Every tool, every project, the same five moves.



Rule of three. If the same bug survives three attempts, go back to the last checkpoint and describe the problem differently. Reverting isn't failure. It's the pro move.



A vague prompt



```
> Build a landing page for a meditation app using React and Tailwind. Make it look nice and modern. Add some animations.
```

What did we leave for the AI to decide?

- Who is it for?
- What does the button do?
- What does "nice" mean?
- What counts as done?

This prompt is not wrong. It just hands every decision to the model. We will see what that produces in a moment.



Words that work better than "nice"

Visual	Motion	Tone	Style
soft, dreamy	smooth, slow, fluid	playful, formal	retro, brutalist
cosy, warm	springy, elastic	gentle, sharp	minimal, maximal
dark, moody	quick, responsive	calming, bold	glassy, grainy

"Calm" becomes useful when you add a palette, a layout, a motion limit, and a reference: "like a Muji store".



The same app, specified



BUILD "STILL", A MEDITATION PAGE FOR BUSY STUDENTS

- PURPOSE** Busy students who want a short break between classes.
- DESIGN** Warm cream, sage green, generous space, gentle motion.
- BEHAVIOUR** Both buttons open a 60-second breathing timer with start, pause, reset, and a completion message.
- CONSTRAINTS** React + Tailwind. Mobile layout. Keyboard access. Reduced motion. No login, backend, or paid services.
- VERIFY** Check both buttons, the timer states, and that mobile has no horizontal overflow.

Purpose, design, behaviour, constraints, verify. The last one is the part everybody skips, and the part that saves you.



Feedback that changes the result

TO CHANGE HOW IT FEELS

"Reduce the hero to one headline and one primary action."

"Use a serif heading for a vintage feel."

"Slow the motion, and respect reduced motion."

"Keep the timer behaviour unchanged."

TO FIX WHAT BROKE

"The submit button does nothing. The console says: [paste error]."

"What are three possible causes? Do not fix anything yet."

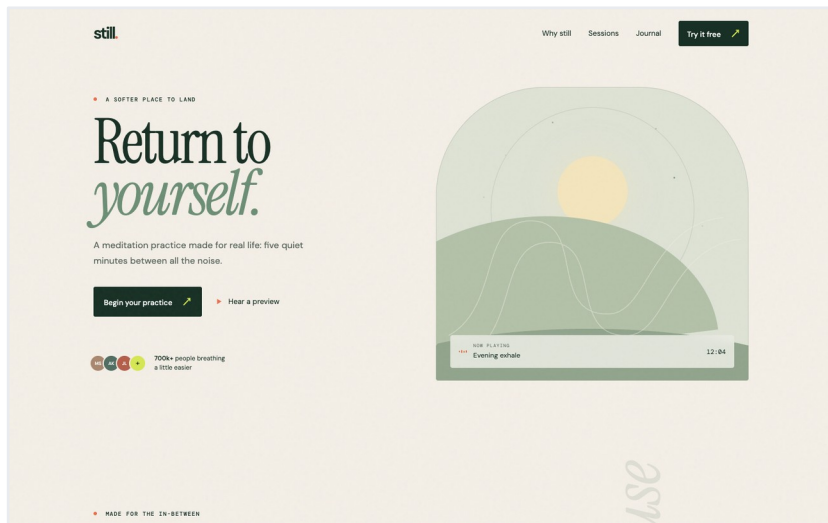
"Add a test that would have caught this."

"Revert to the last version and try a different approach."

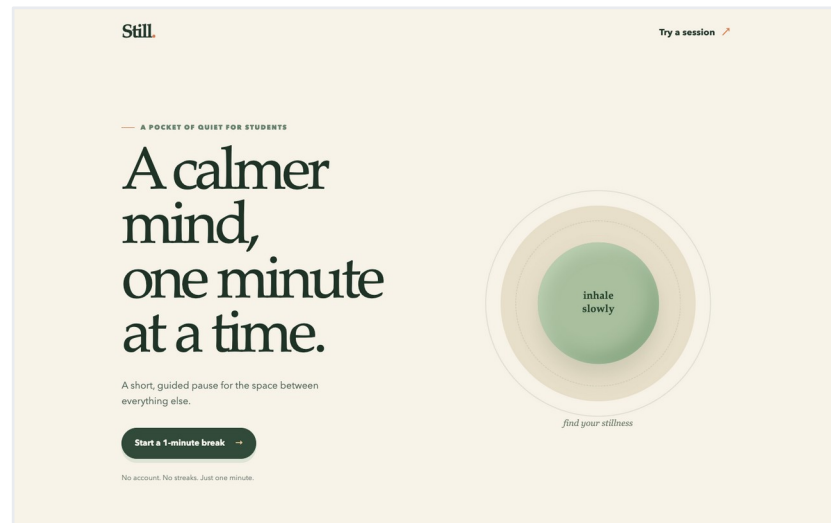
One dimension at a time. Say what to preserve. Say what you see versus what you expected.



Same app, two prompts, same tool



A Vague prompt. Looks great. The buttons are placeholders, and it invented "700k+ people" out of thin air.



B Specific prompt. Less flashy. Both buttons open a working 60-second breathing timer, on mobile, with keyboard access.



Live build: Founder Dinner RSVP

10 : 00

ON THE CLOCK

You shape it. Two things from the room:

- What is the dinner for?
- One question the RSVP form should ask that I wouldn't think of.

QR: PUBLISHED PAGE
appears around minute 7

Scan and RSVP from your phone



What counts as done?

WHAT I CHECKED

- ✓ The form saves, and I saw the row appear
- ✓ It works on a real phone, not just the preview
- ✓ The confirmation says what I asked it to say
- ✓ The admin list shows the count

WHAT I SKIPPED, ON PURPOSE

- ✗ Login
- ✗ Spam protection and rate limits
- ✗ Data privacy: where the emails live, who can read them
- ✗ Backups

Fine for a demo. Fine for a dinner for 40 friends. **Not fine for 400 sign-ups holding people's personal data.**



Five ways this goes wrong

1

It is confidently wrong

It will say "done, this works" when it doesn't. It will invent a number. Trust the screen, not the sentence.

2

Secrets leak

Never paste passwords, API keys, or customer data into a prompt. Vibe-coded apps regularly ship with the database open to the internet.

3

It can destroy things

July 2025: an AI agent on Replit deleted a startup's production database during a code freeze, after being told not to touch anything.

4

The 80 percent cliff

The first 80 percent takes an afternoon. The last 20 percent (edge cases, payments, permissions) is where projects die if nobody understands them.

5

You still own it

March 2025: a founder announced his SaaS was 100 percent vibe-coded. Within days people bypassed his paywall and ran up his API bill. He couldn't fix it.



Plausible output can still be wrong



Invented facts

Does the API, package, number, or citation actually exist?



Contradictions

Do the code, the explanation, and the requirements agree?



Repetition or ambiguity

Does it repeat work, or leave behaviour unclear?



Bias and stereotyping

Who could the wording or the assumptions exclude?



Overgeneralisation

Does a fix for one case quietly break three others?

Five review questions to ask of any output. Run them. Do not accept a generated claim about them.



Cheap habits that prevent expensive days



Checkpoint after every win. Version history, or git.



Ask it to write tests, then make it run them and show you the output.



Ask it to run the app and screenshot it. Not just say it works.



Ask, in these words:

"Is anything here exposed to the public internet that shouldn't be? Check the database rules."



Keep prototype and production apart. Different project, different data, no real customer data.



Before real users, money, or personal data: one engineer reviews login, payments, and data access. One hour.

The human must be in the loop. Prompting is the new syntax. Checking is the new job.



Vibe coding vs vibe engineering

	Vibe coding	Vibe engineering
Focus	Explore a working idea	Deliver maintainable software
AI's role	Generate a draft	Work within a defined task, as a team member
Workflow	Prompt and refine	Specify, build, review, verify, record
Quality	Looks right when tried	Tests and previews show the required behaviour
Context	Mostly in the conversation	Decisions stay with the project: instructions file, notes, git

Same loop. More reading, more checking, and the decisions written down. This is where I work all day.



The delivery workflow, one feature at a time

1

Specify

Human defines the problem and the acceptance checks.

2

Implement

Agent drafts the code and the tests, in a small diff.

3

Review

Human reads the changes and the changed behaviour.

4

Evidence

Tests and a preview expose regressions. The agent never grades its own work.

5

Record

Save the code, the rationale, and the operating notes beside it.

At my desk: plan first, small diffs, tests as the contract, an instructions file that tells the agent the house rules, and a separate check before anything merges.



Tonight. One ugly thing that works.

LEVEL 1

No new account

In ChatGPT, Claude, or Gemini: a single-page tool.

A pitch timer with a 3-minute buzzer.
A split-the-bill calculator. A "should I enter this hackathon" checklist.

LEVEL 2

Browser builder

Lovable, Bolt, or v0.

A landing page with a working waitlist for your idea. Or run both "Still" prompts yourself and compare.

LEVEL 3

You already code

Codex, Claude Code, or Cursor.

Add one feature with a test to a repo you have.

Then: show it to 10 real people this week. Keep every prompt and plan. That pile is your spec.

QR
WORKSHEET

The bottleneck used to be writing code.

Now it's knowing what you want, and checking that you got it.



```
> Ship something ugly this week. |
```



Ask me anything.

Or ask me to build anything.

Conceptual questions get short answers. "Can it do X?" gets built live while we take the next one.

github.com/Ducksss · linkedin.com/in/chai-pin-zheng · [email]

Some slides adapted from Damian Gordon, "Introduction to Vibe Coding and Vibe Engineering" (SlideShare). Quotes: Karpathy, Feb 2025; Collins Dictionary, Nov 2025; Y Combinator, Mar 2025.



Everything from today, one link

